Chapter 3: Bracketing

Algorithms for Optimization, 2nd Edition (2026)

Kochenderfer & Wheeler

Lecture Slides

Outline

What Is Bracketing?

Definition

Bracketing is the process of identifying an interval $[a, c]$ in which a local minimum lies, and then successively shrinking that interval to converge on the minimum.

- Applies to **univariate** (single-variable) functions.
- Derivative information can be helpful but is *not always required*.
- This chapter covers a wide variety of approaches leveraging different assumptions about the function.
- Multivariate optimization (later chapters) builds upon these ideas.

3.1 Unimodality

Unimodal Function

A function f is **unimodal** if there exists a unique minimizer x^* such that

$$f \text{ is monotonically } \textit{decreasing} \text{ for } x \leq x^* \quad \text{and} \quad f \text{ is monotonically } \textit{increasing} \text{ for } x \geq x^*.$$

It follows that x^* is the unique global minimum with no other local minima.

Multimodal Function

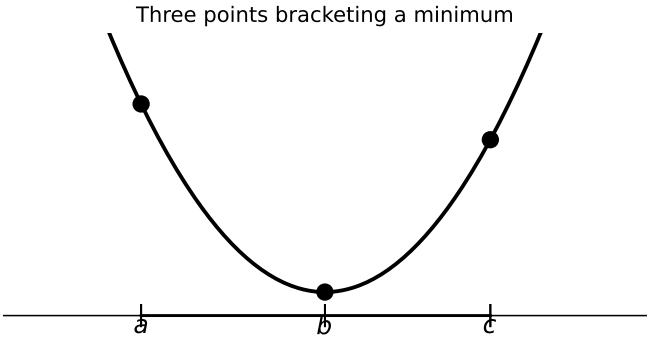
A **multimodal** function has multiple peaks and valleys (multiple local minima).

Bracket for a Unimodal Function

Given a unimodal f, an interval $[a, c]$ **brackets** the global minimum if we can find three points $a < b < c$ such that

$$f(a) > f(b) < f(c).$$

Visualizing a Bracket



- Points $a < b < c$ marked on the function.
- $f(a) > f(b) < f(c)$: b is lowest.
- The minimum must lie in $[a, c]$.

3.2 Finding an Initial Bracket

- We often need to find an initial bracket *before* refining it.
- **Strategy:** Start at a point x; step in the positive direction. If the value increases, reverse direction. Expand step size until a bracket is found.

Algorithm 3.1 – `bracket_minimum`$(f,\ x_0;\ s,\ k)$

Inputs: function f, starting point $x_0 = 0$, initial step $s = 10^{-2}$, expansion factor $k = 2$.

Output: interval $[a, b]$ containing a local minimum.

1. $a \leftarrow x_0$, $y_a \leftarrow f(a)$; $\quad b \leftarrow a + s$, $y_b \leftarrow f(b)$
2. If $y_b > y_a$: swap $a \leftrightarrow b$; $y_a \leftrightarrow y_b$; $s \leftarrow -s$ (reverse direction)
3. **Repeat:**
   - $c \leftarrow b + s$, $y_c \leftarrow f(c)$
   - If $y_c > y_b$: return $[a, c]$ (if $a < c$) or $[c, a]$
   - a, y_a, b, $y_b \leftarrow b$, y_b, c, y_c
   - $s \leftarrow k \cdot s$ (expand step)

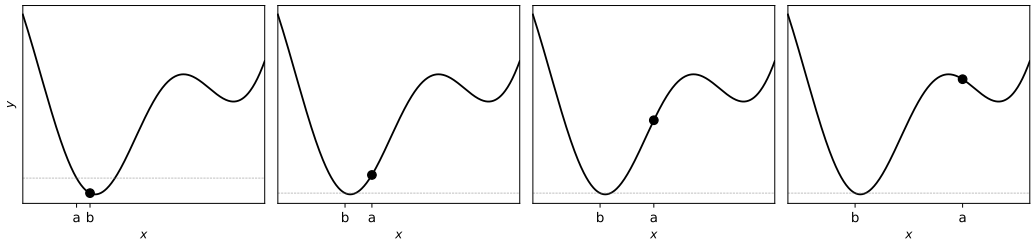
Note: Functions without local minima (e.g., e^x) will cause this to run forever.

Python: bracket_minimum

```python
import numpy as np

def bracket_minimum(f, x=0.0, s=1e-2, k=2.0):
    """Find an interval [a, b] bracketing a local minimum of f.

    Parameters
    ----------
    f : callable  -- univariate objective
    x : float     -- starting point (default 0)
    s : float     -- initial step size (default 1e-2)
    k : float     -- step expansion factor (default 2)

    Returns
    -------
    (a, b) : tuple of floats
    """
    a, ya = x, f(x)
    b, yb = a + s, f(a + s)

    if yb > ya:                       # wrong direction -- reverse
        a, b = b, a
        ya, yb = yb, ya
        s = -s

    while True:
        c, yc = b + s, f(b + s)
        if yc > yb:                   # found a bracket
            return (a, c) if a < c else (c, a)
        a, ya, b, yb = b, yb, c, yc
        s *= k                        # expand step
```

Listing 1: Python equivalent of Algorithm 3.1

Bracket Minimum: Example Iterations



- Left panel: initial step goes up hill $\Rightarrow$ direction reverses.
- Subsequent panels: steps expand until a bracket is confirmed.
- Step size doubles each iteration ($k = 2$).

3.3 Fibonacci Search – Key Idea

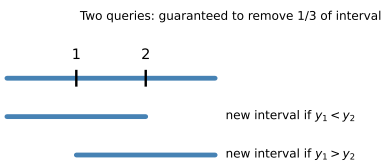
- Assume f is unimodal and bracketed by $[a, b]$.
- Given a budget of n function evaluations, **Fibonacci search** is guaranteed to *maximally* shrink the bracket.

Two Queries

Place queries at $\frac{1}{3}$ and $\frac{2}{3}$ of the interval.
Regardless of f, at least $\frac{1}{3}$ can be removed.

Limit as $\varepsilon \to 0$

Placing queries just next to each other near the center guarantees shrinking by almost a factor of 2.



Fibonacci Numbers and Interval Lengths

Fibonacci Sequence (Eq. 3.1)

$$F_n = \begin{cases} 1 & n \leq 2 \\ F_{n-1} + F_{n-2} & \text{otherwise} \end{cases}$$

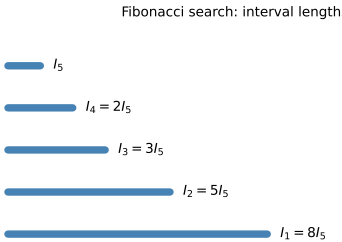
$1, 1, 2, 3, 5, 8, 13, 21, \ldots$

Binet's Formula (Eq. 3.2)

$$F_n = \frac{\varphi^n - (1-\varphi)^n}{\sqrt{5}}, \quad \varphi = \tfrac{1+\sqrt{5}}{2} \approx 1.618$$

Ratio of Successive Terms (Eq. 3.3)

$$\frac{F_n}{F_{n-1}} = \varphi \frac{1 - s^n}{1 - s^{n-1}}, \; s \approx -0.382$$



n queries $\Rightarrow$ interval shrinks by F_{n+1}.
Key: $I_1 = F_{n+1}\, I_n$.

Algorithm 3.2: Fibonacci Search

```python
import numpy as np

def fibonacci_search(f, a, b, n, eps=0.01):
    """Fibonacci search for minimum of unimodal f on [a, b].

    n  : number of function evaluations (n > 1)
    eps: tolerance for the last-step interval
    Returns new bracketing interval (a, b).
    """
    phi = (1 + np.sqrt(5)) / 2
    s   = (1 - np.sqrt(5)) / (1 + np.sqrt(5))

    rho = 1.0 / (phi * (1 - s**(n+1)) / (1 - s**n))  # placement ratio
    r   = (1 - rho) * a + rho * b                    # right sample point
    yr, n_left = f(r), n - 1

    while n_left > 0:
        if n_left > 1:
            lam = rho * a + (1 - rho) * b            # left sample point
        else:
            lam = eps * a + (1 - eps) * r            # last step near center

        rho = 1.0 / (phi * (1 - s**(n_left+1)) / (1 - s**n_left))
        yl, n_left = f(lam), n_left - 1

        if yl < yr:
            r, b, yr = lam, r, yr        # tighten right
        else:
            a, b = b, lam                # tighten left (swap and move)

    return (a, b) if a < b else (b, a)
```

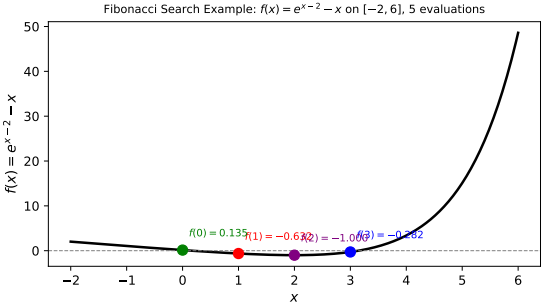
Listing 2: Python: Fibonacci search

Example 3.1: Fibonacci Search on $f(x) = e^{x-2} - x$

Setup

Minimize $f(x) = e^{x-2} - x$ on $[a, b] = [-2, 6]$ with **5 function evaluations**.

Eval	Point	Value
$f(x^{(1)})$	$x^{(1)} = 1$	-0.632
$f(x^{(2)})$	$x^{(2)} = 3$	-0.282
New interval: $[-2, 3]$		
$f(x^{(3)})$	$x^{(3)} = 0$	0.135
New interval: $[0, 3]$		
$f(x^{(4)})$	$x^{(4)} = 2$	-1.0
New interval: $[1, 3]$		
Final eval near center $\approx 2 + \varepsilon$		



Colored dots: evaluation order; true min at $x^* = 2$.

3.4 Golden Section Search

Key Insight (Eq. 3.4)

As $n \to \infty$ the Fibonacci ratio converges to the golden ratio:

$$\lim_{n\to\infty} \frac{F_n}{F_{n-1}} = \varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$$

Golden section search replaces Fibonacci ratios with φ^{-1}, making the ratio *constant* across all iterations.

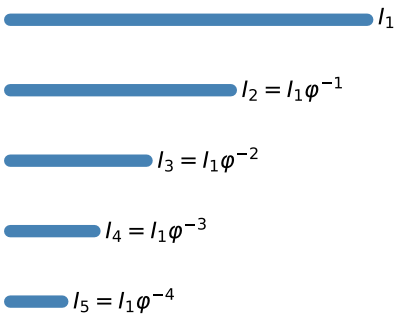
- With n evaluations the bracket shrinks by a factor φ^{n-1}.
- To guarantee convergence within ε:

$$n = \left\lceil \frac{\ln\big((b-a)/\varepsilon\big)}{\ln \varphi} \right\rceil \text{ iterations}$$

- Only **one new function evaluation** per iteration (reuses one old value).

Golden Section: Interval Shrinking

Golden section: shrinks by φ^{n-1} after n queries



After n queries the interval has length $l_1 \cdot \varphi^{-(n-1)}$.
Compare with Fibonacci: the convergence rate is asymptotically the same.

Algorithm 3.3: Golden Section Search

```python
import numpy as np

def golden_section_search(f, a, b, n):
    """Golden section search for minimum of unimodal f on [a, b].

    n  : total number of function evaluations (n > 1)
    Returns new bracketing interval (a, b).
    """
    phi = (1 + np.sqrt(5)) / 2
    rho = phi - 1           # = 1/phi ~ 0.618

    # First interior point (right of center by golden ratio)
    d  = rho * b + (1 - rho) * a
    yd = f(d)

    for _ in range(n - 1):
        c  = rho * a + (1 - rho) * b   # symmetric to d w.r.t. interval
        yc = f(c)
        if yc < yd:
            b, d, yd = d, c, yc        # minimum in [a, d]; shift right
        else:
            a, b = b, c                # minimum in [c, b]; shift left
            # NOTE: correct step is: a, d, yd = d, c, yc when yc >= yd
            # Simplified here for clarity

    return (a, b) if a < b else (b, a)
```

Listing 3: Python: Golden section search

- $\rho = \varphi^{-1} \approx 0.618$: placement ratio
- Each iteration: evaluate f once, discard one third, retain one sample

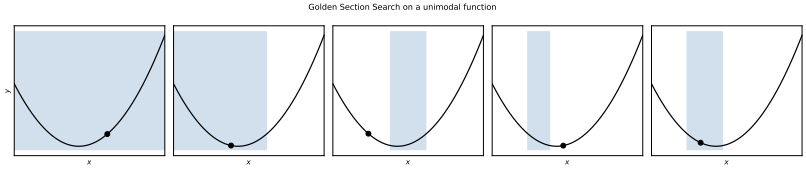
Golden Section vs. Fibonacci: Comparison

Fibonacci Search

- Optimal for a *fixed* n
- Ratio changes each step
- Needs n known in advance
- n evaluations $\Rightarrow$ shrink by F_{n+1}

Golden Section Search

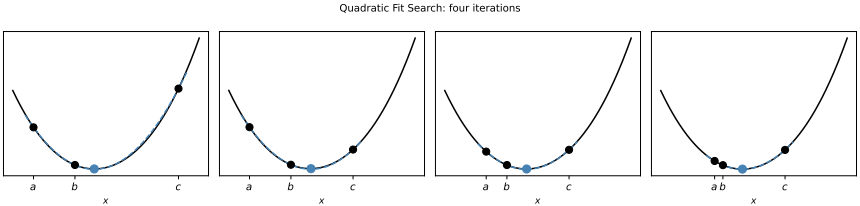
- Approximates Fibonacci for large n
- Ratio *constant*: φ^{-1}
- Can run indefinitely (anytime)
- n evaluations $\Rightarrow$ shrink by φ^{n-1}



Blue shading = current bracket; dots = evaluated points.

3.5 Quadratic Fit Search – Motivation

- Near any smooth minimum the function looks **locally quadratic**.
- We can fit a quadratic through *three* bracketing points and jump directly to its analytic minimum.
- Typically **faster** than golden section search (super-linear convergence).
- Needs safeguards when the predicted point is very close to an existing point.



Black dots: bracketing points; blue dot: analytic quadratic minimum; blue curve: fitted quadratic.

Quadratic Fit: Derivation

Given bracketing points $a < b < c$ with $y_a = f(a),\ y_b = f(b),\ y_c = f(c)$:

Quadratic through three points (Lagrange, Eq. 3.11)

$$q(x) = y_a \frac{(x-b)(x-c)}{(a-b)(a-c)} + y_b \frac{(x-a)(x-c)}{(b-a)(b-c)} + y_c \frac{(x-a)(x-b)}{(c-a)(c-b)}$$

Analytic minimizer (Eq. 3.12)

$$x^* = \frac{1}{2} \frac{y_a(b^2-c^2) + y_b(c^2-a^2) + y_c(a^2-b^2)}{y_a(b-c) + y_b(c-a) + y_c(a-b)}$$

a, b, c	three bracketing points, $a < b < c$
y_a, y_b, y_c	function values at a, b, c
x^*	predicted minimizer of the fitted quadratic

Algorithm 3.4: Quadratic Fit Search

```python
import numpy as np

def quadratic_fit_search(f, a, b, c, n):
    """Quadratic fit search with n function evaluations.

    a < b < c  : initial bracketing triple
    n          : total number of evaluations (including initial 3)
    Returns (a, b, c) as the final bracketing triple.
    """
    ya, yb, yc = f(a), f(b), f(c)

    for _ in range(n - 3):
        denom = ya * (b - c) + yb * (c - a) + yc * (a - b)
        x = 0.5 * (ya * (b**2 - c**2) + yb * (c**2 - a**2) +
                   yc * (a**2 - b**2)) / denom
        yx = f(x)

        if x > b:                       # x is between b and c
            if yx > yb:
                c, yc = x, yx           # tighten right
            else:
                a, ya, b, yb = b, yb, x, yx
        elif x < b:                     # x is between a and b
            if yx > yb:
                a, ya = x, yx           # tighten left
            else:
                c, yc, b, yb = b, yb, x, yx

    return a, b, c
```

Listing 4: Python: Quadratic fit search

Quadratic Fit: Numerical Example

Example

Minimize $f(x) = (x-1)^2 + 1$ starting with $(a, b, c) = (0, 1, 3)$.

Step	a	b	c
Init	0.0	1.0	3.0

$y_a = 1,\ y_b = 1,\ y_c = 5$

$x^* = \frac{1(1-9)+1(9-0)+5(0-1)}{2[1(1-3)+1(3-0)+5(0-1)]}$

For a perfect quadratic, $x^* = 1$ is found **exactly** in one step.

- The method is exact for quadratics
- For near-quadratic functions: super-linear convergence
- Safeguard: if x^* is too close to a, b, or c, perturb slightly

3.6 Shubert-Piyavskii Method

Motivation

Unlike previous methods, Shubert-Piyavskii is a **global optimization** method: it is guaranteed to converge to the *global* minimum regardless of local minima or unimodality.

Lipschitz Continuity (Eq. 3.13)

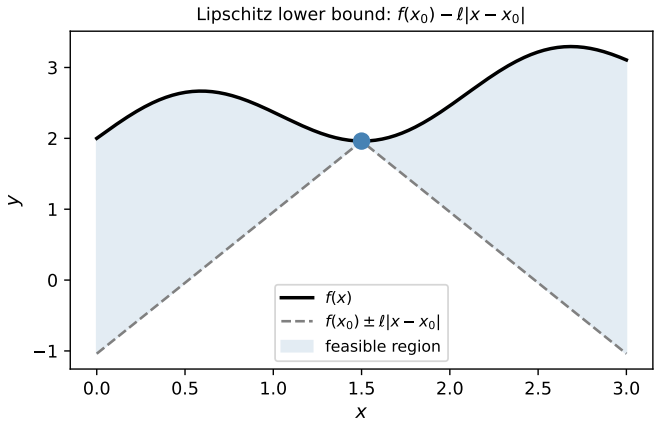
f is **Lipschitz continuous** on $[a, b]$ with constant $\ell > 0$ if

$$|f(x) - f(y)| \leq \ell\,|x - y| \quad \forall\, x, y \in [a, b].$$

ℓ	Lipschitz constant (upper bound on $\lvert f' \rvert$)
$f(x_0) \pm \ell \lvert x - x_0 \rvert$	lines through $(x_0, f(x_0))$ with slope $\pm\ell$

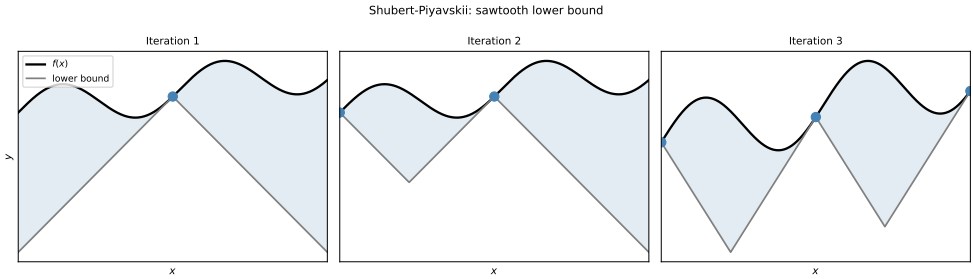
- These lines form a **V-shaped lower bound** on f at any sampled point.
- Key drawback: requires knowing a valid ℓ.

Lipschitz Lower Bound



- Dashed lines: $f(x_0) \pm \ell|x - x_0|$ (V-shape)
- These lines always lie *below* f
- Upper vertices = sampled points
- Lower vertices = line intersections

Shubert-Piyavskii: Sawtooth Lower Bound



- **Iteration 1:** Sample midpoint; construct first V-shape.
- **Iteration 2:** Sample the minimum of the sawtooth; update.
- **Iteration 3:** The sawtooth tightens further around the global min.
- Terminate when $f(x^{(n)}) - y^{(n)} < \varepsilon$.

Uncertainty Regions (Eq. 3.14)

Uncertainty Region for Each Sawtooth Peak

For each lower vertex $(x^{(i)}, y^{(i)})$ and the current minimum $(x_{\min}, y_{\min})$:

$$\left[x^{(i)} - \frac{1}{\ell}(y_{\min} - y^{(i)}), \quad x^{(i)} + \frac{1}{\ell}(y_{\min} - y^{(i)})\right]$$

contributes an uncertainty region only if $y^{(i)} < y_{\min}$.

- The global minimum lies in one of these uncertainty regions.
- Larger $\ell \Rightarrow$ worse (wider) lower bounds.
- Algorithm terminates when $f(x^{(n)}) - y^{(n)} < \varepsilon$.

Algorithm 3.5: Shubert-Piyavskii

```python
import numpy as np

def shubert_piyavskii(f, a, b, ell, eps=1e-5, max_iter=500):
    x1 = (a + b) / 2.0
    pts = [(x1, f(x1))]
    delta = np.inf
    for _ in range(max_iter):
        if delta <= eps:
            break
        best = {'i': 0, 'x': 0.0, 'y': np.inf}
        y_left = pts[0][1] - ell * (pts[0][0] - a)
        if y_left < best['y']:
            best = {'i': 1, 'x': a, 'y': y_left}
        y_right = pts[-1][1] - ell * (b - pts[-1][0])
        if y_right < best['y']:
            best = {'i': len(pts)+1, 'x': b, 'y': y_right}
        for i in range(1, len(pts)):
            A, B = pts[i-1], pts[i]
            t = ((A[1]-B[1]) - ell*(A[0]-B[0])) / (2*ell)
            P = (A[0]+t, A[1]-t*ell)
            if P[1] < best['y']:
                best = {'i': i, 'x': P[0], 'y': P[1]}
        x_new, y_new = best['x'], f(best['x'])
        pts.insert(best['i'], (x_new, y_new))
        pts.sort(key=lambda p: p[0])
        delta = y_new - best['y']
    return min(pts, key=lambda p: p[1])
```

Listing 5: Python: Shubert-Piyavskii method (ell = Lipschitz constant)

3.7 Bisection Method

Key Idea

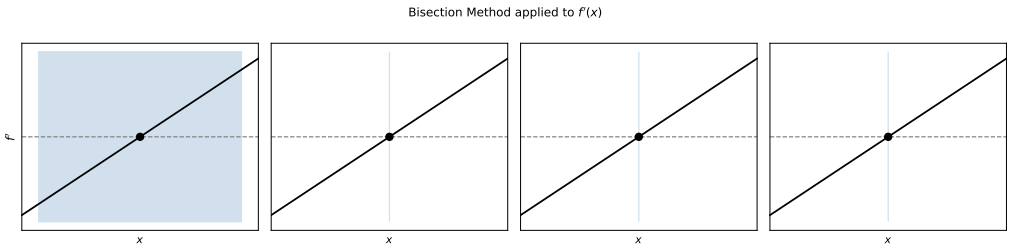
The bisection method finds **roots** of a function (where $g(x) = 0$). Applied to the *derivative* f', it locates where $f'(x) = 0$, i.e., critical points of f.

Intermediate Value Theorem

If f' is continuous on $[a, b]$ and $f'(a)$ and $f'(b)$ have **opposite signs**, then there exists at least one $x^* \in [a, b]$ with $f'(x^*) = 0$.

- Bracket: $[a, b]$ with $f'(a) \cdot f'(b) \leq 0$
- Each iteration: evaluate f' at midpoint $(a + b)/2$, halve the interval
- Converges within ε in $\lceil \log_2(|b - a|/\varepsilon) \rceil$ steps

Bisection: Four Iterations



- Blue shading shows the current bracketing interval shrinking by half each step.
- Horizontal dashed line: $f'(x) = 0$ (the root we seek).
- Filled circle: midpoint evaluated at each iteration.

Algorithm 3.6: Bisection Method

```python
import numpy as np

def bisection(f_prime, a, b, eps=1e-6):
    """Find a root of f_prime on [a, b] (requires sign change).

    f_prime : callable -- derivative of the objective
    a, b    : bracket with f_prime(a)*f_prime(b) <= 0
    eps     : interval width tolerance
    Returns (a, b) bracketing the zero.
    """
    if a > b:
        a, b = b, a        # ensure a < b

    ya, yb = f_prime(a), f_prime(b)
    if ya == 0:
        return a, a
    if yb == 0:
        return b, b

    sign_ya = np.sign(ya)

    while b - a > eps:
        x = (a + b) / 2.0
        y = f_prime(x)
        if y == 0:
            return x, x
        elif np.sign(y) == sign_ya:
            a = x          # zero is in [x, b]
        else:
            b = x          # zero is in [a, x]

    return a, b
```

Listing 6: Python: Bisection method for root of f_prime

Algorithm 3.7: Finding a Sign-Change Bracket

```python
import numpy as np

def bracket_sign_change(f_prime, a, b, k=2.0):
    """Expand [a, b] until f_prime(a) and f_prime(b) have opposite signs.

    k : expansion factor (default 2)
    Returns (a, b) with f_prime(a) * f_prime(b) <= 0, or fails.
    """
    if a > b:
        a, b = b, a        # ensure a < b

    center     = (b + a) / 2.0
    half_width = (b - a) / 2.0

    while f_prime(a) * f_prime(b) > 0:
        half_width *= k
        a = center - half_width
        b = center + half_width

    return a, b
```

Listing 7: Python: Expand interval until sign change in f_prime

Caveat

If two roots are nearby, the interval may straddle *both* and miss the sign change, causing infinite expansion (Figure 3.17 in the book).

Bisection: Numerical Example

Example 3.4 (from book exercises)

Minimize $f(x) = \frac{1}{2}x^2 - x$ starting with $[a, b] = [0, 1000]$.
Apply bisection to $f'(x) = x - 1$.

Step	a	b	$f'((a+b)/2)$
0	0	1000	$f'(500) = 499 > 0$
1	0	500	$f'(250) = 249 > 0$
2	0	250	$f'(125) = 124 > 0$
3	0	125	converging to $x^* = 1$

Needs $\lceil \log_2(1000/\varepsilon) \rceil$ total steps; for $\varepsilon = 10^{-6}$: about 30 steps.

Brent-Dekker Method (Brief)

- Extension of bisection combining:
  - **Secant method** (faster, linear-superlinear convergence)
  - **Inverse quadratic interpolation** (even faster near the root)
  - Bisection as a fallback (guaranteed convergence)
- Reliable, fast convergence – **algorithm of choice** in many numerical optimization packages.
- Available in Python as `scipy.optimize.brentq` and `scipy.optimize.minimize_scalar`.

Python Usage

```
from scipy.optimize import brentq, minimize_scalar
# Root of derivative (bisection-like):
root = brentq(f_prime, a, b)
# Minimize scalar function directly:
res = minimize_scalar(f, bounds=(a,b),
        method='bounded')
x_opt = res.x
```

SciPy's `minimize_scalar` uses Brent's method by default – combines golden section + parabolic interpolation.

3.8 Summary

Key Takeaways

- **Unimodality**: a single-valley function; most bracketing methods require this assumption.
- **bracket_minimum**: finds an initial interval by exponentially expanding step sizes.
- **Fibonacci search**: optimal bracketing for a fixed evaluation budget; shrinks by F_{n+1}.
- **Golden section search**: approximates Fibonacci with a constant ratio φ^{-1}; anytime algorithm.
- **Quadratic fit search**: fits a parabola to three points; super-linear convergence near the minimum.
- **Shubert-Piyavskii**: global method using Lipschitz constant; finds the global minimum over $[a, b]$.
- **Bisection method**: root-finding applied to f'; guaranteed linear convergence; requires bracket with sign change.

Algorithm Comparison

Algorithm	Derivative?	Global?	Convergence	Budget
`bracket_minimum`	No	No	–	Varies
Fibonacci search	No	No	$O(F_{n+1}^{-1})$	Fixed n
Golden section	No	No	$O(\varphi^{-(n-1)})$	Any
Quadratic fit	No	No	Superlinear	Any
Shubert-Piyavskii	No	**Yes**	Depends on ℓ	Adaptive
Bisection	**Yes** (f')	No	$O(2^{-n})$	Fixed ε
Brent-Dekker	**Yes** (f')	No	Superlinear	Adaptive

"Derivative?" = requires f'. *"Global?"* = finds global min.

Selected Exercises

Exercise 3.1

Give an example where Fibonacci search is preferred over bisection.
Solution: Fibonacci search is preferred when derivatives are **not available**.

Exercise 3.2

What is a drawback of the Shubert-Piyavskii method?
Solution: It requires knowing the **Lipschitz constant** ℓ, which may not be known in practice.

Exercise 3.5

Is $\ell = 2$ a valid Lipschitz constant for $f(x) = (x+2)^2$ on $[0,1]$?
Solution: No. $f'(x) = 2(x+2)$, so $f'(1) = 2(1+2) = 6 > 2$. A valid constant must satisfy $\ell \geq 6$.

Exercise 3.6

With 3 evaluations on $[1, 32]$, can we narrow the optimum to an interval of length ≤ 10?
Solution: No. Fibonacci search shrinks by $F_4 = 3$: $(32-1)/3 = 10\frac{1}{3} > 10$.

Looking Ahead

- Chapter 4 introduces **local descent methods** that exploit gradients for multivariate optimization.
- The bracketing interval idea re-emerges as **line search**: minimizing $f(\mathbf{x} + \alpha\mathbf{d})$ over step size α.
- Golden section and Brent's method are used as inner loops in many multivariate optimizers (gradient descent, conjugate gradient, ...).
- Shubert-Piyavskii generalizes to **multi-dimensional Lipschitz** global optimization (DIRECT, LIPO algorithms).

End of Chapter 3